Ruh.ai Platform Product Requirements Document

Digital employee operating system built on OpenClaw gateway runtimes

Document status	Internal draft for product, engineering, and solutions leadership
Version / date	v1.0	7 March 2026

Product decision in one sentence

Ruh.ai should not be a collection of bespoke agents. It should be the horizontal platform that creates, deploys, governs, and measures digital employees, while using OpenClaw as the runtime plane for sessions, routing, tools, channels, sub-agents, hooks, and cron [1][2][5][8][9].

Executive summary

Dimension	Decision
Platform category	Digital employee operating system built on OpenClaw runtime gateways.
Primary outcome	Create, deploy, supervise, and improve collaborating digital employees without bespoke engineering for every customer.
Hard constraint	One OpenClaw gateway per tenant trust boundary; Ruh.ai must not treat one shared gateway as a multi-tenant bus.
Go-to-market pattern	Horizontal core platform plus vertical workflow packs, with construction as the first industry pack.

Why this architecture

OpenClaw already gives Ruh.ai the gateway runtime primitives that matter: the gateway is the source of truth for sessions, routing, and channel connections; each agent has isolated workspace/auth/session state; tools are typed and policy-controlled; sub-agents and ACP sessions exist for delegation; hooks and cron exist for automation [1][2][5][8][9]. The missing product is the enterprise control plane on top of those primitives.

1. Purpose of this PRD

This document defines the product requirements for the **Ruh.ai digital employee platform**, a system that lets teams create, deploy, supervise, and improve cooperating digital employees inside customer environments. The document treats OpenClaw as the agent gateway/runtime operating layer and specifies the control plane, packaging model, governance, and collaboration model Ruh.ai must build above it.

The intended outcome is a repeatable product platform that serves **many construction companies** and later other industries, without requiring a new bespoke agent architecture per customer. Domain differences should be absorbed by packs - reusable bundles of templates, task contracts, connectors, dashboards, and evals - rather than by rebuilding the platform each time.

2. Product goals and non-goals

2.1 Goals

- Allow a tenant admin or solution architect to define a new digital employee from a reusable template in less than 30 minutes.
- Compile each employee into a governed runtime bundle: workspace files, config, skills, hooks, cron jobs, credentials, and policy.
- Support collaboration between employees through typed tasks, shared artifacts, and approval gates rather than ad hoc cross-chat.
- Make deployment repeatable across multiple customers through pack abstractions instead of one-off prompt projects.
- Provide auditability for every run, tool call, approval, and system write-back.

2.2 Non-goals

- Ruh.ai will not be a generic “chat with documents” product.
- Ruh.ai will not replace the customer’s system of record (ERP, CRM, Procore, Jira, GitHub, etc.).
- Ruh.ai will not use a single OpenClaw host as a shared runtime for mutually untrusted tenants or business units.
- V1 will not support unrestricted public skill installation into production runtimes.
- V1 will not allow fully autonomous irreversible actions without policy or human approval.

3. Product principles

[] **Trust boundaries first:** Security architecture comes before workflow convenience. OpenClaw itself documents a trusted-operator model, so Ruh.ai must explicitly build trust-bounded deployments [6][7].

[] **Roles, not prompts:** A digital employee is a governed role contract: inputs, outputs, tools, permissions, KPIs, and approvals - not just a prompt.

[] **Artifacts over transcripts:** Business work should resolve into structured artifacts and system updates, not only chat replies.

[] **Typed collaboration:** Employee-to-employee handoffs should be task-based and schema-driven; raw session-to-session messages should be a debugging tool, not the default enterprise operating model [5][8].

[] **Packs over bespoke builds:** The core platform stays horizontal; domain packs provide templates, contracts, connectors, dashboards, and evals.

[] **Human control at irreversible steps:** The platform should automate analysis and preparation aggressively, and require approval at material write-backs, payments, external sends, and policy exceptions.

4. System architecture

Ruh.ai must use a **two-plane architecture**: **OpenClaw runtime plane** for actual agent execution, and **Ruh.ai control plane** for enterprise management. This separation is not optional. OpenClaw is explicit that it is personal/trusted-operator by default, not a shared multi-tenant bus [6][7]. Therefore multi-customer scale must come from the control plane and from trust-bounded runtime deployment.

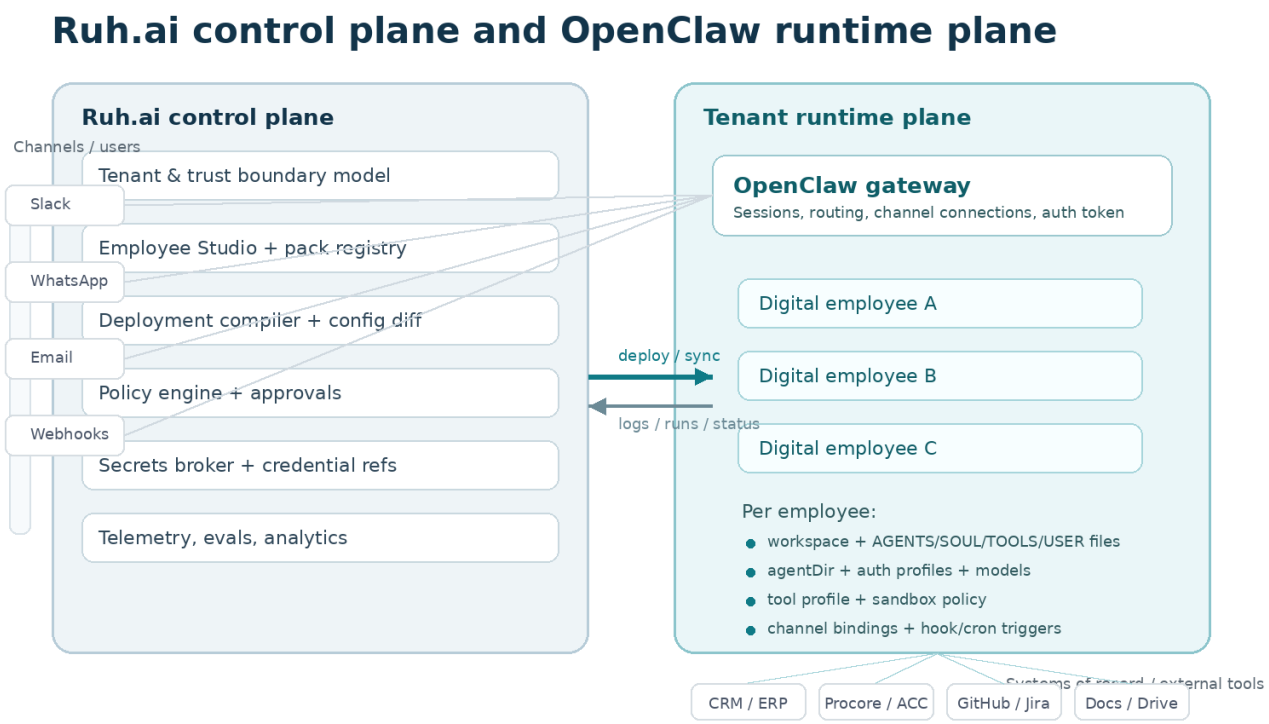

Figure 1. Recommended separation between Ruh.ai control plane and tenant runtime planes.

4.1 What stays in OpenClaw vs. what Ruh.ai must build

Layer	Owned by	Responsibility
Gateway runtime	OpenClaw	Sessions, routing, channel connections, workspace loading, per-agent auth, tools, sub-agents, hooks, cron, browser/nodes/canvas [1][2][5][8][9].
Control plane	Ruh.ai	Tenant model, deployment compiler, employee studio, pack registry, policy engine, secrets broker, task graph, approvals, evals, observability, analytics, billing.
Systems of record	Customer	ERP, CRM, project systems, ticketing, email, drives, source control, or other operational systems that remain the source of business truth.

Hard design requirement

A trust boundary is a first-class product object. Every production deployment must resolve to a dedicated runtime boundary, such as a separate gateway container, VM, OS user, or customer-managed host. Ruh.ai should enforce this in the product, not leave it as an implementation convention [6][7].

4.2 Requirements inherited from OpenClaw

- Separate gateway per trust boundary. If users may be adversarial, use separate gateways or separate OS/users/hosts; session isolation alone is not enough [6][7].
- Do not rely on workspace paths as a sandbox. OpenClaw notes that the workspace is the default cwd, not a hard sandbox unless sandboxing is enabled [2][7].
- Treat plugins and custom tool packages as trusted code. Only allow curated, pinned, reviewed packages in production [6][7].
- Use secure DM mode for shared inboxes. DM scope should isolate contexts in multi-user deployments [3][6][11].
- Deny control-plane tools on untrusted surfaces. gateway, cron, sessions_spawn, and sessions_send should be denied by default where outside input can reach them [6].
- Treat webhook and hook payloads as untrusted input. Automation triggers need the same policy discipline as human messages [6][9].

5. Core domain model

The platform should be modeled around **reusable business objects** rather than loose agent configs. These objects let Ruh.ai support multiple customers and multiple domain packs on the same core architecture.

5.1 Core entities

Entity	Purpose
Tenant	Customer account owning packs, environments, usage, and billing.
Trust Boundary	Isolated runtime boundary mapped to one or more dedicated OpenClaw gateways.
Environment	Dev, staging, or production slice within a trust boundary.
Pack	Reusable bundle of templates, contracts, connectors, evals, dashboards, and policies.
Employee Template	Versioned definition of a digital employee role before deployment.
Employee Deployment	Compiled and activated runtime instance of a template in a target environment.
Workspace Kit	Generated OpenClaw workspace files plus Ruh-specific manifests and policies.
Tool Profile	Named allow/deny policy mapped to OpenClaw tools and plugin tools.
Task Contract	Typed input/output schema that defines work an employee can accept or emit.
Task	Runtime work item with SLA, owner, status, artifacts, and audit trail.
Artifact	Document, draft, record update, summary, file, or structured output attached to a task.
Approval Policy	Rules determining which side effects require review, who can approve, and escalation logic.
Credential Binding	Least-privilege mapping from an employee to a secrets broker reference.
Run	Single execution instance with transcript, tool calls, tokens, timing, cost, and result.
Eval Suite	Regression, quality, and safety tests used before promotion or pack release.

5.2 Mapping Ruh.ai objects to OpenClaw primitives

Ruh.ai object	OpenClaw primitive	Notes
Trust Boundary	Dedicated gateway host / container / OS user	This is the enterprise isolation layer required on top of OpenClaw [6][7].
Employee Deployment	agents.list[] entry	Each employee gets its own workspace, agentDir, session store, auth, tool policy, and sandbox [2][7].
Workspace Kit	workspace directory	Contains AGENTS.md, SOUL.md, TOOLS.md, USER.md, IDENTITY.md, HEARTBEAT.md, optional MEMORY.md, and pack files [10].
Channel Presence	bindings[] + channel config	Route inbound channel traffic to the correct employee [1][2][11].
Tool Profile	tools.* and agents.list[].tools	Translate Ruh allow/deny profiles into OpenClaw tool policy [5][7].
Credential Binding	agent auth profile / env injection / broker token	Credentials are per agent and should not be copied around casually [2][7].
Task Trigger	hooks / webhooks / cron	Use hooks for events and cron for time-based wakeups [9].
Internal Delegation	sessions_spawn / sub-agents / ACP sessions	Use for bounded delegated execution, not as the entire collaboration layer [8].

6. Digital employee lifecycle

OpenClaw workspaces are file-based and load operating instructions from files such as AGENTS.md, SOUL.md, TOOLS.md, USER.md, IDENTITY.md, HEARTBEAT.md, and optional MEMORY.md [10]. Ruh.ai should turn that into a governed lifecycle so that every employee is repeatable, testable, and deployable.

1. Author or clone an Employee Template from a pack in Employee Studio.
2. Configure the role contract: objective, accepted task contracts, output artifacts, model profile, tool profile, memory policy, approval policy, and KPI owner.
3. Attach skills, plugin tools, connectors, and credential bindings. Every package must be version-pinned and policy-approved before promotion to production.
4. Compile the template into a runtime bundle. Ruh.ai generates workspace files, config fragments, bindings, hooks, cron specs, and policy manifests.
5. Run offline evals plus dry-run integration checks against the target environment.
6. Deploy into dev/staging/prod and activate channels or automation triggers.
7. Observe runs, tune policies, compare versions, and promote or roll back as needed.

Compiled bundle output

For every deployment, the compiler should output: (1) workspace files, (2) config fragments for agents, bindings, tools, hooks, and cron, (3) package lockfile for skills/hooks/plugins, (4) credential reference manifest, (5) task-contract schema bundle, and (6) approval/eval metadata.

7. Collaboration model for digital employees

The platform must answer a key product question: **how do digital employees work together safely?** The answer should be: through typed tasks, shared artifacts, policy gates, and bounded delegation. OpenClaw sub-agents and session tools are useful building blocks, but the platform should wrap them inside a business-safe collaboration model [5][8].

Figure 2. Recommended collaboration model: task graph outside, delegated sub-agents inside.

7.1 Collaboration rules

Area	Requirement
Within one employee	Use OpenClaw sub-agents or ACP sessions for bounded parallel work, long-running research, or specialized code execution [8].
Between employees	Use Ruh.ai tasks, artifacts, approvals, and allowlisted handoff graphs. Cross-employee chat should not be the primary business protocol.
Shared state	Store shared truth in work items, artifacts, and target systems - not in private session memory alone.
Escalation	Employees may emit escalation tasks when required context, permissions, or confidence thresholds are missing.
Thread UX	Where the underlying adapter supports threads, Ruh.ai may preserve a user-friendly threaded experience, but the system of record remains the task graph [8].

7.2 Example task contracts

Contract	Input	Output	Side effects
triage_case	ticket / email / webhook payload	classified case, suggested owner, next step	none
draft_external_message	case file, context pack, policy	draft email/message + rationale	approval required
create_record	validated payload, target system binding	record id, link, receipt	approval or policy gate
research_and_summarize	question, context set, sources	summary artifact, evidence list	none
exception_escalation	failure, reason, context	escalation task, SLA clock	none

Platform rule

Do not make raw employee-to-employee chat the default enterprise protocol. Instead, require task contracts with

schemas, artifact attachments, deadlines, and explicit allowed handoffs. This keeps collaboration observable, reviewable, and compatible with approvals, SLAs, and retries.

8. Detailed functional requirements

9.1 Tenant, environment, and deployment

FR-001 [P0] Trust boundary creation: The system shall allow creation of a tenant with one or more trust boundaries, each mapped to dedicated OpenClaw runtime infrastructure.

FR-002 [P0] Environment separation: The system shall support development, staging, and production environments with separate deployments, credentials, and audit scopes.

FR-003 [P0] Deployment compiler: The system shall compile an employee template into workspace files, config fragments, bindings, hooks, cron specs, and policy manifests.

FR-004 [P0] Versioned rollout: The system shall version every deployment and support progressive rollout, rollback, and diffing between deployed versions.

FR-005 [P0] Pre-flight validation: Before activation, the system shall validate required models, connectors, credentials, packages, and policy compatibility.

FR-006 [P1] Runtime health and drift: The system shall surface gateway health, channel status, sync status, and config drift for every deployment.

9.2 Employee Studio and pack system

FR-010 [P0] Template authoring: The system shall provide a template editor for role definition, accepted contracts, outputs, owners, and metadata.

FR-011 [P0] Definition-as-code: The system shall support a human-readable employee definition language with schema validation and Git-friendly diffing.

FR-012 [P0] Pack composition: The system shall group templates, contracts, connectors, dashboards, and evals into reusable packs.

FR-013 [P1] Inheritance and override: A tenant shall be able to extend a base pack without forking the entire core runtime configuration.

FR-014 [P0] Preview and test: Template authors shall be able to preview compiled workspace files and generated config before deployment.

FR-015 [P1] Change history: The system shall maintain version history, promotion status, and changelog per template and pack.

9.3 Skills, tools, connectors, and credentials

FR-020 [P0] Private registry: Production runtimes shall install skills, hooks, and plugins only from a curated, pinned, reviewed private registry.

FR-021 [P0] Tool profiles: The system shall expose named tool profiles that compile to OpenClaw tool allow/deny rules and per-agent overrides.

FR-022 [P0] Connector abstraction: The system shall support connectors for channels, email/webhooks, file stores, and external systems through plugin tools or adapter services.

FR-023 [P0] Credential binding: Credentials shall be bound per employee and per environment with least privilege and rotation support.

FR-024 [P1] Dry-run connector tests: Admins shall be able to validate a connector and credential binding without enabling business-side effects.

FR-025 [P0] Package approval: Installing or updating a plugin, hook pack, or skill pack shall require policy approval in production environments.

FR-026 [P1] Trigger assets: Hooks, webhooks, and cron schedules shall be first-class assets linked to templates and packs.

9.4 Collaboration, workflow, and approvals

FR-030 [P0] Task graph engine: The system shall create and track work items, tasks, dependencies, owners, due times, SLAs, and status transitions.

FR-031 [P0] Typed task contracts: Every employee-facing task shall declare an input schema, output schema, artifact requirements, and escalation behavior.

FR-032 [P0] Collaboration allowlist: The system shall define which employees may hand off work to which other employees, by contract type.

FR-033 [P1] Bounded internal delegation: Employees shall be able to use sub-agents or ACP sessions for bounded delegated work with depth, timeout, and concurrency limits.

FR-034 [P0] Shared artifact store: The system shall store drafts, evidence, receipts, summaries, files, and write-backs as artifacts attached to work items.

FR-035 [P0] Approval gates: High-risk side effects shall require policy-based approval before the action is executed.

FR-036 [P1] Retries and idempotency: The workflow engine shall support retries, compensation paths, and idempotency keys for actions against external systems.

FR-037 [P1] Inbox and queue operations: Supervisors shall have queue views for pending tasks, stuck runs, approvals, and SLA breaches.

9.5 Memory, context, and knowledge

FR-040 [P0] Workspace kit generation: The compiler shall generate OpenClaw workspace files and Ruh-specific manifests for every deployment.

FR-041 [P0] Context assembly: At task start, the system shall assemble a context pack from artifacts, records, memory snippets, and policy relevant to that contract.

FR-042 [P0] Memory write policy: The system shall define what can be written to persistent employee memory, what stays ephemeral, and what is prohibited.

FR-043 [P0] Shared versus private memory: Shared/task memory and private employee memory shall be stored separately so sensitive context is not leaked across work scopes.

FR-044 [P1] Search and retrieval: The system shall support search across artifacts, prior runs, and tenant knowledge sources, with source attribution.

FR-045 [P1] Retention controls: Admins shall be able to configure retention, deletion, and redaction rules for runs, artifacts, and memory.

9.6 Governance, security, and compliance

FR-050 [P0] Role-based access control: The control plane shall define admin, designer, reviewer, operator, auditor, and service roles with least privilege.

FR-051 [P0] Approval policy engine: Approvals shall be configurable by action type, connector, data sensitivity, risk score, or employee role.

FR-052 [P0] Trust-bounded deployment: The platform shall prevent deployment of mutually untrusted tenants into the same runtime boundary.

FR-053 [P0] Per-employee sandboxing: Sandbox mode, scope, and session visibility shall be configurable per employee, with secure defaults for public or untrusted surfaces.

FR-054 [P0] Secret isolation: Raw credentials shall not be shared between employees by default; secret access shall be auditable and revocable.

FR-055 [P0] Complete audit trail: Every run, tool call, approval, config change, package install, and system delivery shall be logged with actor and timestamp.

FR-056 [P0] Sensitive tool defaults: gateway, cron, sessions_spawn, and sessions_send shall be denied by default on employee surfaces that process untrusted input.

FR-057 [P1] Incident controls: Admins shall be able to pause an employee, revoke bindings, quarantine a pack, or disable a gateway quickly.

FR-058 [P1] Hybrid deployment model: The platform shall support customer-managed or VPC-hosted gateways when enterprise policy requires local control of runtime data.

9.7 Observability, evaluation, and operations

FR-070 [P0] Run timeline: Operators shall see a step-by-step execution timeline including context bundle, tool calls, model usage, outputs, and errors.

FR-071 [P1] Replay and diff: The system shall support transcript replay and version-to-version comparison for debugging regressions.

FR-072 [P1] Offline eval suites: Templates and packs shall support regression, quality, and safety eval suites that run before promotion.

FR-073 [P0] SLO metrics: The system shall track queue time, completion time, success rate, approval latency, side-effect failure rate, and cost per task.

FR-074 [P0] Health monitoring: The system shall monitor gateway reachability, channel status, hook/cron health, plugin status, and connector failures.

FR-075 [P1] Alerting: Operators shall receive alerts for stuck runs, SLA breaches, cost spikes, missing credentials, and policy violations.

FR-076 [P1] Exportable reports: Audit, usage, and operational reports shall be exportable by tenant, pack, employee, and date range.

9.8 API and extensibility

FR-080 [P1] Control-plane APIs: The platform shall expose APIs for tenants, packs, templates, deployments, tasks, artifacts, approvals, and evals.

FR-081 [P1] Pack SDK / CLI: Pack authors shall have a CLI/SDK to lint, test, publish, and promote packs.

FR-082 [P1] Event bus: The platform shall emit standard events such as TaskAssigned, RunStarted, ApprovalRequested, ArtifactCreated, RunFailed, and DeploymentSynced.

FR-083 [P1] Data portability: Tenants shall be able to export templates, contracts, task history, and audit records in documented formats.

FR-084 [P1] Model/router abstraction: The platform shall support configurable model providers and routing strategies without rewriting employee definitions.

9. Non-functional requirements

Area	Requirement
Security	Hard tenant isolation at the gateway boundary, encryption in transit/at rest, signed packages, and auditable secret access.

Availability	Control plane target 99.9% monthly availability; runtime plane should degrade gracefully when a channel or connector fails.
Performance	Median task dispatch < 2 seconds after trigger arrival; compile-preview < 10 seconds for standard templates.
Scalability	Support thousands of runs/day per tenant without cross-tenant resource contention.
Recoverability	Deployment rollback < 5 minutes; task and run history preserved across gateway restarts.
Observability	Structured logs, metrics, traces, and exportable audit events for every task and tool execution.
Cost control	Per-tenant budgets, model routing policies, run-level cost visibility, and sub-agent concurrency limits.

10. MVP scope and phased delivery

The first release should prove that Ruh.ai is a **horizontal product platform** with one reference industry pack. It should not attempt to solve every workflow. The MVP must instead make employee creation, deployment, collaboration, approvals, and observability repeatable across customers.

10.1 MVP release scope

Area	What must ship
Foundation	Trust boundary model; runtime deployment manager; code-mode template editor; pack registry v1; private skill/plugin registry; audit log pipeline.
Employee definition	Employee definition language; workspace compiler; tool profiles; credential bindings; channel bindings; policy manifests.
Workflow core	Task contracts; task graph; assignment rules; artifact store; human approvals; retry/idempotency primitives.
Operations	Run timeline; transcript replay; deployment diff/rollback; gateway health dashboard; cost and token reporting.
Construction pack alpha	Reference pack with 3-4 digital employees and shared task contracts to prove the horizontal platform model.

Suggested rollout sequence

Release 1: trust boundaries, deployment manager, employee definition language, task graph, approvals, audit log. Release 2: pack SDK, offline evals, analytics, multi-environment promotion. Release 3: richer dashboards, policy simulation, private pack exchange, and deeper connector coverage.

11. Risks and open questions

- Will the control plane be SaaS-only, hybrid, or fully self-hostable for enterprise customers?
- What is the preferred secrets pattern: direct auth-profile materialization, short-lived broker tokens, or plugin-mediated vault access?
- How much workflow logic belongs in Ruh.ai orchestration versus hooks/cron running at the gateway edge?
- What is the minimum safe public surface for customers who want inbound email/webhook automation at scale?
- How should eval suites be versioned and promoted alongside packs and employee templates?

Appendix A. Example employee definition

The example below shows how a human-readable template can describe a digital employee. Ruh.ai should store this as versioned source, validate it against a schema, and compile it into OpenClaw runtime assets.

```yaml
apiVersion: ruh.ai/v1alpha1
kind: EmployeeTemplate
metadata:
  pack: construction-core
  id: document-controller
  version: 0.3.0
spec:
  objective: >
    Turn inbound project documents into classified tasks, drafts, and system updates.
  accepts:
    - triage_case
    - research_and_summarize
    - draft_external_message
  emits:
    - create_record
    - exception_escalation
  openclaw:
    model: anthropic/claude-sonnet-4-5
    thinking: medium
    tools:
      profile: messaging
      allow: [message, pdf, web_fetch, sessions_spawn]
      deny: [gateway, cron, exec, process]
    sandbox:
      mode: all
      scope: agent
    subagents:
      allowAgents: [document-researcher]
      maxConcurrent: 4
  channels:
    - slack:#project-ops
    - email:shared-inbox
  memory:
    mode: task-plus-role
    writePolicy: explicit
  approvals:
    draft_external_message: required
    create_record: policy
  connectors:
    - sharepoint
    - procore
  evalSuites:
    - dc-regression-v1
    - external-message-safety-v1
```

Appendix B. Example compiled OpenClaw fragment

```yaml
agents:
  list:
    - id: document-controller
      workspace: /var/ruh/t1/prod/document-controller/workspace
      tools:
        profile: messaging
        allow: [message, pdf, web_fetch, sessions_spawn]
        deny: [gateway, cron, exec, process]
      sandbox:
        mode: all
        scope: agent
      subagents:
        allowAgents: [document-researcher]
        maxConcurrent: 4
bindings:
  - agentId: document-controller
    match:
      channel: slack
      accountId: ops
      peer: { kind: channel, id: C123456 }
  - agentId: document-controller
    match:
      channel: gmail
```

```
      accountId: shared-inbox
hooks:
  enabled: true
cron:
  enabled: true
```

Appendix C. Reference notes

OpenClaw capability references used in this document. All references were checked on 7 March 2026 and come from official OpenClaw documentation or the official GitHub security pages.

- [1] OpenClaw Docs home/overview: https://docs.openclaw.ai/
- [2] OpenClaw Multi-Agent Routing: https://docs.openclaw.ai/concepts/multi-agent
- [3] OpenClaw Session Management: https://docs.openclaw.ai/concepts/session
- [4] OpenClaw Skills: https://docs.openclaw.ai/tools/skills
- [5] OpenClaw Tools: https://docs.openclaw.ai/tools
- [6] OpenClaw Security: https://docs.openclaw.ai/gateway/security
- [7] OpenClaw Multi-Agent Sandbox & Tools and GitHub Security Overview: https://docs.openclaw.ai/tools/multi-agent-sandbox-tools ; https://github.com/openclaw/openclaw/security
- [8] OpenClaw Session Tools, Sub-Agents, ACP Agents: https://docs.openclaw.ai/concepts/session-tool ; https://docs.openclaw.ai/tools/subagents ; https://docs.openclaw.ai/tools/acp-agents
- [9] OpenClaw Hooks and Cron Jobs: https://docs.openclaw.ai/automation/hooks ; https://docs.openclaw.ai/automation/cron-jobs
- [10] OpenClaw Setup, Agent Workspace, Memory: https://docs.openclaw.ai/start/openclaw ; https://docs.openclaw.ai/concepts/agent-workspace ; https://docs.openclaw.ai/concepts/memory
- [11] OpenClaw Configuration Examples: https://docs.openclaw.ai/gateway/configuration-examples